Epistemic Uncertainty in LLM Hallucinations

CENG463 – Technical Checkpoint Progress Report

May 11, 2026

Group Members

Ata TürkYılmaz	300201055
Berkhan Kargılı	290201108
Atakan Kumbaracı	300206012

https://github.com/TAtaTrkylmz/CENG463-FinalProject

1. Problem Definition

Task. This project investigates whether *epistemic uncertainty signals* extracted from large language models (LLMs) can reliably detect hallucinated outputs. Given a question and a candidate answer, the system must classify the answer as factual (label 0) or hallucinated (label 1). The task is therefore a **binary classification** problem framed around natural language understanding and model introspection.

Importance. Hallucinations – cases where a language model generates fluent but factually incorrect content – represent one of the most critical reliability problems in deployed NLP systems. Detecting them automatically is essential for safe deployment in high-stakes domains such as medicine, law, and education. Rather than relying on external fact-checking, this project explores whether the model's own uncertainty (expressed via token-level log-probabilities) can serve as an intrinsic hallucination signal, a direction that is both computationally efficient and broadly applicable.

Dataset, Input, and Output. The project uses the **HaluEval QA** benchmark. Each original sample contains a question, a knowledge snippet, a factual answer, and a hallucinated answer. During preprocessing, each sample is expanded into two supervised rows, giving a balanced binary classification dataset. The system input is a (question, candidate answer) pair – optionally augmented with the knowledge snippet as context – and the output is a binary hallucination label.

Technical Challenges. The task is technically challenging for several reasons. First, hallucinated answers are often fluent and superficially plausible, making surface-level detection difficult. Second, uncertainty signals derived from small local models (e.g., `distilgpt2`) may not transfer reliably to the larger models that produce hallucinations in practice. Third, distinguishing genuine epistemic uncertainty from aleatoric uncertainty (inherent ambiguity in the question) is a non-trivial research problem. Finally, the binary construction of HaluEval introduces its own biases that must be understood during evaluation.

2. Dataset and Preprocessing Status

- **Dataset name and source:** HaluEval QA, available via the Hugging Face `datasets` library (`pminervini/HaluEval`, split `qa`).
- **Sample construction:** Each original HaluEval record yields two supervised examples – one factual (label 0) and one hallucinated (label 1) – resulting in a balanced binary dataset.
- **Features / input type:** Text fields: `question`, `answer` (candidate), and optionally `knowledge` (the reference snippet used in context-mode inference).
- **Target label:** Binary integer – 0 (factual) or 1 (hallucinated).
- **Train / validation / test split:** Produced deterministically by `scripts/prepare_dataset.py`. Splits are written as JSONL files under `data/processed/halueval_qa/`.
- **Preprocessing steps completed:**
  - Download and normalisation via Hugging Face `datasets`.
  - Expansion into two supervised rows per original sample.

  - Deterministic train / val / test split and serialisation to JSONL.
  - Processed splits verified to exist under `data/processed/halueval_qa/`.
- **Known issues:** The dataset is artificially balanced by construction, which may not reflect real-world hallucination rates. Downstream metrics should be interpreted with this in mind. No missing-value or encoding issues were observed.

Status: The dataset has been downloaded, normalised, split, and is ready for experiments.

3. Literature Review Progress

The literature review is ongoing. The following thematic areas and representative works have been surveyed:

Hallucination detection and benchmarking. HaluEval (Li et al., 2023) provides the dataset backbone of this project and frames hallucination detection as a supervised classification task using human-annotated factual and hallucinated answer pairs. TruthfulQA (Lin et al., 2022) offers a complementary evaluation angle, measuring how often models generate false beliefs. These works motivate the use of a supervised binary classification setup rather than an open-ended generation evaluation.

Uncertainty quantification in neural language models. Seminal work on predictive uncertainty (Gal & Ghahramani, 2016; Lakshminarayanan et al., 2017) established Bayesian and ensemble-based approaches to uncertainty estimation. More recently, Kuhn et al. (2023) and Kadavath et al. (2022) adapted these ideas to autoregressive LLMs by using sequence-level log-probabilities and verbalized self-confidence as uncertainty proxies, which directly inspired the entropy-based baseline in this project.

Retrieval-augmented generation (RAG) and context faithfulness. Lewis et al. (2020) demonstrated that grounding generation in retrieved documents reduces hallucinations. The RAG-compare baseline in this project operationalises this finding: it measures the log-probability delta between a memory-only prompt and a context-augmented prompt as a hallucination signal.

Lexical and shallow baselines. TF-IDF with linear classifiers remains a strong baseline for text classification (Joachims, 1998) and provides an interpretable lower bound for the uncertainty-based methods.

Planned direction. Based on this review, the project will compare lexical, entropy-based, and RAG-based signals. A potential improvement is to replace `distilgpt2` with a stronger instruction-tuned model to obtain more calibrated log-probabilities.

4. Baseline Models

Three baselines have been selected and fully implemented:

1. **Lexical SVM** (`lexical_svm`). A TF-IDF vectoriser over the concatenated question and answer, fed into a linear Support Vector Machine. This baseline is intentionally

shallow: it captures surface-level lexical overlap that may correlate with hallucination (e.g., vague or unusually specific phrasing) without any LM involvement. It serves as the interpretable lower bound.

2. **Entropy Classifier** (`entropy`). Token-level log-probabilities are extracted from a local causal LM (`distilgpt2`) running in *memory mode* (no retrieved context). Aggregate features – mean log-probability, total sequence entropy, and token count – are passed to a logistic regression classifier. This baseline directly operationalises the epistemic-uncertainty hypothesis: a model that is uncertain about the tokens in a hallucinated answer should assign lower probabilities to those tokens.
3. **RAG Compare** (`rag_compare`). The same local LM scores each answer twice – once without context (memory mode) and once with the HaluEval knowledge snippet prepended (context mode). The delta in mean log-probability between the two modes is thresholded to produce a binary prediction. A factual answer is expected to receive a larger probability boost from relevant context than a hallucinated one.

All three baselines are implemented in `src/llm_uncertainty/baselines.py` and are runnable via `scripts/run_baseline.py`. Preliminary results are reported in Section 5.

5. Initial Experimental Results

Initial experiments were conducted on a small subset (80 training / 20 validation samples) to verify the end-to-end pipeline. Results are summarised in Table 1.

Table 1: Preliminary validation results (small-scale run, $n_{\text{val}} = 20$).

Model	**Accuracy**	**Precision**	**Recall**	**F1**	**Status**
Lexical SVM	–	–	–	–	Implemented; full-data run pending
Entropy	–	–	–	–	Implemented; full-data run pending
RAG Compare	–	–	–	–	Implemented; full-data run pending

Note: Numerical metrics are not yet finalised because the small-scale run was used solely to validate pipeline correctness (data flow, feature extraction, and output serialisation). The `make_report_assets.py` script has been verified to aggregate metrics and generate plots into `reports/tables/` and `reports/figures/`. Full-dataset runs are scheduled as the immediate next step (see Section 10).

Despite the absence of consolidated numbers at this stage, the pipeline produces consistent predictions and metric JSON files for each baseline, confirming that the experimental infrastructure is sound.

6. Planned Improvements and Technical Direction

The next phase will proceed in three directions:

Scale up experiments. Run all three baselines on the full HaluEval QA dataset (removing the `-limit` flag) to obtain stable, reportable metrics.

Stronger LM backbone. Replace `distilgpt2` with a larger instruction-tuned model (e.g., `GPT-2 medium` or an open-weight instruction model) to obtain better-calibrated log-probabilities for the entropy and RAG-compare baselines. This directly tests whether epistemic uncertainty scales with model capacity.

Expanded uncertainty features. Beyond mean log-probability and sequence entropy, add token-level variance, top-k probability mass, and calibration metrics (Expected Calibration Error, Brier score) as features for the entropy classifier.

Threshold optimisation for RAG Compare. The current delta-thresholding approach uses a fixed threshold. A calibrated threshold search on the validation set will be introduced to improve precision-recall balance.

Addressing potential class imbalance. Although HaluEval is balanced by construction, experiments on larger or out-of-domain slices may introduce imbalance. Oversampling (SMOTE) or class-weighted loss will be evaluated if needed.

7. Ablation and Error Analysis Plan

Ablation study. The following components will be ablated in the final report:

- *Context vs. memory prompting:* Removing the knowledge snippet from the RAG-compare pipeline to measure the contribution of retrieved context.
- *Feature subsets for entropy classifier:* Evaluating subsets of log-probability features (mean only vs. mean + variance vs. full feature set) to understand which uncertainty dimensions are most predictive.
- *LM backbone size:* Comparing `distilgpt2` against larger models to quantify the effect of model capacity on uncertainty signal quality.

Error analysis. Model failures will be examined through:

- *Confusion matrices:* To identify asymmetric error patterns (false positives vs. false negatives).
- *Qualitative inspection:* Manual review of misclassified examples to identify recurring failure modes (e.g., hallucinations that are nearly paraphrases of factual answers).
- *Confidence-error correlation:* Cases where the model assigns high confidence to a wrong prediction will be flagged as overconfident failures and examined separately.
- *Difficulty stratification:* Errors will be stratified by question type and answer length to identify subgroup weaknesses.

8. Visualization and Interpretation

The following visualisations are planned and partially scaffolded in `src/llm_uncertainty/reporting.p`

- **Confusion matrices** for each baseline, to expose false-positive and false-negative patterns.
- **ROC and Precision-Recall curves** to compare the three baselines under varying decision thresholds, especially relevant for the RAG-compare method.
- **Calibration plots** (reliability diagrams) to assess whether predicted probabilities align with empirical accuracy, supporting the epistemic-uncertainty narrative.
- **Log-probability distribution plots** contrasting the score distributions for factual vs. hallucinated answers under memory and context modes.
- **Learning curves** to monitor overfitting and data efficiency of the entropy classifier as training size grows.
- **t-SNE or UMAP projections** of TF-IDF and log-probability feature spaces to visually inspect the separability of factual vs. hallucinated samples.

All figure outputs are saved to `reports/figures/` via `make_report_assets.py`.

9. GitHub and Reproducibility Status

- **Repository:** https://github.com/TAtaTrkylmz/CENG463-FinalProject
- **Commit history:** 8 commits as of the report date; consistent development documented.
- **README:** Present and describes project goals, default dataset, setup, and a complete step-by-step end-to-end reproduction workflow.
- **Dependency file:** `requirements.txt` is present and pins all major dependencies: `pandas`, `numpy`, `scikit-learn`, `pyyaml`, `tqdm`, `matplotlib`, `seaborn`, `datasets`, `evaluate`, `transformers`, `torch`.
- **Repository structure:**

```
CENG463-FinalProject/
|-- data/
|   |-- raw/                    # Reserved for raw downloads
|   |-- external/               # External resources placeholder
|   +-- processed/halueval_qa/  # Train / val / test JSONL splits
|-- docs/
|   |-- papers/                 # Literature review storage
|   +-- progress_report_template.md
|-- reports/
|   |-- figures/                # Generated plots
|   +-- tables/                 # Metrics CSV (initial_results.csv)
|-- results/
|   |-- generations/            # Scored JSONL (memory & context)
|   |-- metrics/                # Per-baseline metrics JSON
|   +-- predictions/            # Prediction CSVs
|-- scripts/
|   |-- prepare_dataset.py
|   |-- run_inference.py
|   |-- run_baseline.py
|   |-- run_rag.py
```

```
|   |-- evaluate.py
|   +-- make_report_assets.py
|-- src/llm_uncertainty/
|   |-- baselines.py
|   |-- data.py
|   |-- features.py
|   |-- local_lm.py
|   |-- metrics.py
|   |-- reporting.py
|   |-- prompts.py
|   +-- io.py
|-- .gitignore
|-- README.md
|-- progress.md
+-- requirements.txt
```

Reproduction instructions:

1. `pip install -r requirements.txt`
2. `python scripts/prepare_dataset.py -limit 100`
3. `python scripts/run_inference.py -input data/processed/halueval_qa/train.jsonl \`
   `-output results/generations/halueval_qa_train_memory.jsonl -mode memory -limit 80`
4. `python scripts/run_baseline.py -baseline entropy ...`
5. `python scripts/make_report_assets.py`

10. Current Challenges and Next Steps

Current challenges:

- *LM scoring speed:* Extracting token-level log-probabilities for every sample with a local causal LM is slow on CPU. Running the full dataset currently requires either GPU access or significant wall-clock time.
- *Model quality vs. availability:* `distilgpt2` is fast and freely available but is not instruction-tuned, meaning its log-probabilities may not faithfully reflect semantic hallucination. A stronger model requires more compute.
- *Metric consolidation:* Small-sample runs have been used only for pipeline validation; final numeric results are not yet stable.
- *Literature review depth:* The docs/papers/ folder is a placeholder; written summaries of reviewed papers remain to be committed.

Next steps before final submission:

1. Run all baselines on the full HaluEval QA dataset and record stable metrics.
2. Evaluate a larger or instruction-tuned LM backbone for the entropy and RAG-compare baselines.
3. Expand uncertainty features (ECE, Brier score, token-level variance).

4. Complete ablation experiments and document findings.
5. Perform qualitative error analysis and produce all planned visualisations.
6. Commit written literature review summaries to `docs/papers/`.
7. Increase commit frequency to maintain a consistent development record.